

PORTFOLIO - MONSTER ADVENTURE

GENNEMGANG AF FOERSTE ITERATION

Foerste iteration implementerer et simpelt terminalspil med en karakter, en liste af egne monstre og kampe mod fjendtlige monstre. Spillet gemmer ikke data mellem koerelser, hvilket passer til kravene for iterationen.

Udviklingsflowet er holdt objektorienteret: foerst er domoeneklasserne for monster og karakter oprettet, derefter kataloget med tilgaengelige monstre, kampmotoren og til sidst terminalmenuerne.

Nuvaerende status:

- * Spilleren kan oprette en ny karakter med et navn.
- * En ny karakter starter med to Hest.
- * Karakteren kan have op til fire monstre.
- * Spilleren kan starte et eventyr, vaelge en fjende og vaelge et levende monster til kampen.
- * Kampen vaelger tilfaeldigt hvem der angriber foerst, hvorefter angrebene skifter.
- * Hvis et eget monster besejres, og karakteren stadig har levende monstre, sendes et nyt ind mod samme fjende.
- * Hvis fjenden besejres, kan spilleren beholde den.
- * Hvis karakteren allerede har fire monstre, kan spilleren udskifte et eksisterende monster.
- * Hvis alle egne monstre besejres, vender spillet tilbage til hovedmenuen.
- * Spilleren kan lukke spillet fra hovedmenuen.

BESKRIVELSE AF NUVAERENDE SYSTEM

Systemet er et C++17 terminalprogram uden eksterne biblioteker. 'Game' styrer hovedmenuen og eventyret. 'Input' validerer brugerinput. 'MonsterCatalog' indeholder de tilgaengelige monstertyper. 'Character' ejer spillerens monstre, mens 'Battle' afvikler en kamp mellem to 'Monster'-objekter.

USE CASE DIAGRAM

flowchart LR

```
Player["Spiller"]
Create["Lav ny karakter"]
View["Vis karakter"]
Adventure["Start eventyr"]
ChooseEnemy["Vaelg fjende"]
ChooseMonster["Vaelg eget monster"]
Fight["Kaemp"]
Capture["Behold besejret monster"]
Replace["Udskift monster"]
Exit["Luk spillet"]
```

Player --> Create

Player --> View
Player --> Adventure
Adventure --> ChooseEnemy
Adventure --> ChooseMonster
Adventure --> Fight
Fight --> Capture
Capture --> Replace
Player --> Exit

Use cases:

- * Lav ny karakter: Spilleren indtaster et navn, og karakteren starter med to Hest.
- * Vis karakter: Spilleren ser karakterens monstre, hp og styrke.
- * Start eventyr: Spilleren kan vælge at kæmpe eller vende tilbage.
- * Vælg fjende: Spilleren vælger et monster fra monsterlisten.
- * Vælg eget monster: Spilleren vælger et levende monster fra karakterens liste.
- * Kæmp: Systemet afvikler kampen med tilfældig første angriber og skiftende ture.
- * Behold besejret monster: Spilleren kan tage en besejret fjende.
- * Udskift monster: Hvis karakteren har fire monstre, kan spilleren erstatte et monster.
- * Luk spillet: Programmet afsluttes.

DOMAIN MODEL

```
classDiagram
class Character {
    name
    monsters
}

class Monster {
    name
    hitPoints
    maxHitPoints
    strength
}

class MonsterCatalog {
    monsters
}

class Battle {
    randomEngine
}

Character "1" o-- "0..4" Monster
MonsterCatalog "1" o-- "*" Monster
Battle ..> Monster
```

SEKVENSDIAGRAMMER

Lav ny karakter

```
sequenceDiagram
    actor Spiller
    participant Game
    participant Input
    participant Character
    participant MonsterCatalog

    Spiller->>Game: Vaelger "Lav ny karakter"
    Game->>Input: Laes navn
    Input-->>Game: Navn
    Game->>Character: Opret karakter
    Game->>MonsterCatalog: Hent starter Hest
    Game->>Character: Tilfoej Hest
    Game->>Character: Tilfoej Hest
    Game-->>Spiller: Karakter oprettet
```

Kamp

```
sequenceDiagram
    actor Spiller
    participant Game
    participant MonsterCatalog
    participant Character
    participant Battle
    participant Monster

    Spiller->>Game: Vaelger kamp
    Game->>MonsterCatalog: Vis fjender
    Spiller->>Game: Vaelger fjende
    Game->>Character: Vis levende monstre
    Spiller->>Game: Vaelger eget monster
    Game->>Battle: fight(egetMonster, fjende)
    Battle->>Monster: takeDamage()
    Battle->>Monster: takeDamage()
    Battle-->>Game: BattleResult
    Game-->>Spiller: Resultat
```

Behold besejret monster

```
sequenceDiagram
    actor Spiller
```

participant Game
participant Character

```
Game-->>Spiller: Tilbyder besejret monster
Spiller->>Game: Svarer ja
alt Under fire monstre
    Game->>Character: addMonster(monster)
else Fire monstre
    Game-->>Spiller: Vaelg om der skal udskiftes
    Spiller->>Game: Vaelger monster
    Game->>Character: replaceMonster(index, monster)
end
```

UML - CLASS DIAGRAM

classDiagram

```
class Game {
    +run()
    -showMainMenu()
    -createNewCharacter()
    -startAdventure()
    -offerCapturedMonster(monster)
}
```

```
class Input {
    +readIntInRange(prompt, min, max)
    +readNonEmptyLine(prompt)
    +readYesNo(prompt)
    +waitForEnter()
}
```

```
class Character {
    +maxMonsterCount
    +name()
    +monsters()
    +addMonster(monster)
    +replaceMonster(index, monster)
    +hasLivingMonsters()
    +livingMonsterIndexes()
}
```

```
class Monster {
    +name()
    +hitPoints()
    +maxHitPoints()
    +strength()
    +isDefeated()
    +takeDamage(damage)
    +healToFull()
    +summary()
}
```

```

}

class MonsterCatalog {
    +monsters()
    +starterHorse()
}

class Battle {
    +fight(playerMonster, enemyMonster, output)
}

```

```

Game o-- Input
Game o-- MonsterCatalog
Game o-- Character
Game ..> Battle
Character o-- Monster
MonsterCatalog o-- Monster
Battle ..> Monster

```

GIT-LOG

Git-log kan indsattes i PDF'en med:

```
git log --oneline --decorate
```

GIT BRANCHING STRATEGY

I denne foerste iteration bruges 'master' som stabil hovedgren, fordi projektet er lille og arbejdet kun omfatter en enkelt samlet prototype. Ved senere iterationer bruges korte feature branches med navne som 'feature/save-game' eller 'feature/tests', som merges tilbage i 'master', naar de bygger og opfylder kravene.

BACKLOG

- * Gem og indlaes karakterer mellem spilkoersler.
- * Flere eventyrtyper og svaerhedsgrader.
- * Bedre balance for hp og styrke.
- * Mulighed for at helbrede monstre via en menu.
- * Automatiske tests af domoenklasser og kampregler.